

Exploratory Analysis

López-Chablé Edgard Uriel

Contents

1. Import files and preprocess data	1
2. Exploratory data analysis	4
Session information	6

```
# Load necessary packages
library(cowplot)
library(viridis)
library(tximport)
library(DESeq2)
library(tidyverse)
library(ggrepel)

# TIFF output directory (created automatically)
tiff_dir <- "../results/TIFFs/1_ExploratoryAnalysis"
dir.create(tiff_dir, showWarnings = FALSE, recursive = TRUE)
```

1. Import files and preprocess data

1.1. Import count files (Kallisto)

We set the path and pattern for the Kallisto files to load the counts.

```
files <- list.files(full.names = TRUE,
                    path = "../data/counts",
                    pattern = "*.tsv")

files

## [1] "../data/counts/C1.tsv" "../data/counts/C2.tsv" "../data/counts/T1.tsv"
## [4] "../data/counts/T2.tsv"
```

1.2. Import experimental metadata

We load the metadata, which contains the conditions and replicates for our experiment.

```
metadatos <- read.csv(file = "../data/metadata_all.csv", row.names = 1)
head(metadatos)
```

```
##      Condition Replicates batch
## C1   Control      C1_R1      1
## C2   Control      C1_R2      2
## T1 Treatment      T1_R1      1
## T2 Treatment      T1_R2      2
```

Quality check: We verify if the number of count files matches the number of samples in our metadata.

```
length(files) == nrow(metadata)
```

```
## [1] TRUE
```

1.3. Build the tx2gene object

Since we are working with bacteria (*Pectobacterium*), there is no complex alternative splicing. Because of this, the transcript name is the same as the gene name.

Note: We read the first count file to get the gene IDs without using absolute local paths.

```
kallisto_ejemplo <- read.delim(files[1])

tx2gene <- data.frame(
  TXNAME = kallisto_ejemplo$target_id,
  GENEID = kallisto_ejemplo$target_id
)
```

1.4. Import with tximport and create DESeq2 object

We merge the counts and set up the DESeqDataSet object, using Condition as our main experimental variable.

```
txi.kallisto <- tximport(files,
  type = "kallisto",
  tx2gene = tx2gene)
```

Note: importing 'abundance.h5' is typically faster than 'abundance.tsv'

```
## reading in files with read_tsv
```

```
## 1 2 3 4
## summarizing abundance
## summarizing counts
## summarizing length
```

```
dds <- DESeqDataSetFromTximport(txi.kallisto,
  colData = metadata,
  design = ~ Condition)
```

```
## Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in
## design formula are characters, converting to factors
```

```
## using counts and average transcript lengths from tximport
```

```
dds
```

```
## class: DESeqDataSet
## dim: 4572 4
## metadata(1): version
## assays(2): counts avgTxLength
## rownames(4572): lcl|NZ_JBCFOM010000001.1_cds_AAGW04_RS00515_104
## lcl|NZ_JBCFOM010000001.1_cds_AAGW04_RS01695_335 ...
## lcl|NZ_JBCFOM010000085.1_cds_WP_342414474.1_4391
## lcl|NZ_JBCFOM010000086.1_cds_WP_342414475.1_4392
## rowData names(0):
## colnames(4): C1 C2 T1 T2
## colData names(3): Condition Replicates batch
```

1.5. Extract and filter counts

To improve our statistical results and reduce noise, we remove genes with low expression. We only keep genes that have an average of more than 10 counts across all samples.

```
# 1. Extract and save raw counts
```

```
rawcounts <- as.matrix(counts(dds))
head(rawcounts)
```

```
##
## lcl|NZ_JBCFOM010000001.1_cds_AAGW04_RS00515_104 249 283 318 299
## lcl|NZ_JBCFOM010000001.1_cds_AAGW04_RS01695_335 3837 3910 2898 2188
## lcl|NZ_JBCFOM010000001.1_cds_AAGW04_RS01865_369 1519 1384 1726 1552
## lcl|NZ_JBCFOM010000001.1_cds_WP_002208627.1_289 453 489 511 530
## lcl|NZ_JBCFOM010000001.1_cds_WP_005971891.1_35 16632 17007 8011 6078
## lcl|NZ_JBCFOM010000001.1_cds_WP_005973933.1_458 5530 5815 4249 4091
```

```
write.csv(rawcounts, file = "../results/rawcounts.csv", row.names = TRUE)
```

```
# 2. Filter low expression genes
```

```
filtcounts <- rawcounts[rowMeans(rawcounts) > 10, ]
head(filtcounts)
```

```
##
## lcl|NZ_JBCFOM010000001.1_cds_AAGW04_RS00515_104 249 283 318 299
## lcl|NZ_JBCFOM010000001.1_cds_AAGW04_RS01695_335 3837 3910 2898 2188
## lcl|NZ_JBCFOM010000001.1_cds_AAGW04_RS01865_369 1519 1384 1726 1552
## lcl|NZ_JBCFOM010000001.1_cds_WP_002208627.1_289 453 489 511 530
## lcl|NZ_JBCFOM010000001.1_cds_WP_005971891.1_35 16632 17007 8011 6078
## lcl|NZ_JBCFOM010000001.1_cds_WP_005973933.1_458 5530 5815 4249 4091
```

```
# 3. Calculate the percentage of genes kept
```

```
p <- (nrow(filtcounts) / nrow(rawcounts)) * 100
paste("Expressed genes kept after filtering =", round(p, 2), "%")
```

```
## [1] "Expressed genes kept after filtering = 99.87 %"
```

```
write.csv(filtcounts, file = "../results/filtcounts.csv", row.names = TRUE)
```

We update our DESeqDataSet object to only use the genes we kept.

```
expgen <- rownames(filtcounts)
dds <- dds[expgen, ]
dds

## class: DESeqDataSet
## dim: 4566 4
## metadata(1): version
## assays(2): counts avgTxLength
## rownames(4566): 1c1|NZ_JBCFOM010000001.1_cds_AAGW04_RS00515_104
## 1c1|NZ_JBCFOM010000001.1_cds_AAGW04_RS01695_335 ...
## 1c1|NZ_JBCFOM010000085.1_cds_WP_342414474.1_4391
## 1c1|NZ_JBCFOM010000086.1_cds_WP_342414475.1_4392
## rowData names(0):
## colnames(4): C1 C2 T1 T2
## colData names(3): Condition Replicates batch
```

2. Exploratory data analysis

2.1. Variance Stabilizing Transformation (VST)

We apply the VST to normalize the data. This makes the variance independent of the mean, which helps clustering methods like PCA work better.

```
vsd <- vst(dds, blind = FALSE)
```

```
## using 'avgTxLength' from assays(dds), correcting for library size
```

2.2. Principal Component Analysis (PCA)

We calculate and plot the PCA to see how similar the samples are and check if they group by their experimental condition.

```
# Get coordinates and explained variance
pcaData <- plotPCA(vsd, intgroup = "Condition", returnData = TRUE)

## using ntop=500 top features by variance

percentVar <- round(100 * attr(pcaData, "percentVar"))

paste("Total explained variance (PC1 + PC2) =", sum(percentVar), "%")

## [1] "Total explained variance (PC1 + PC2) = 99 %"
```

Create and save the PCA plot:

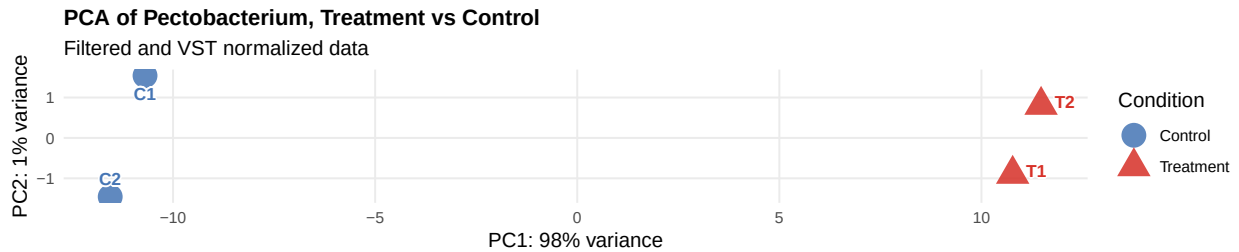
```

# Consistent palette: blue = Control, red = Treatment
pca_palette <- c("Control" = "#4575B4", "Treatment" = "#D73027")

pca <- ggplot(pcaData,
  aes(x = PC1, y = PC2,
    color = Condition, shape = Condition)) +
  geom_point(size = 7, alpha = 0.85) +
  # circle (16) for Control, filled triangle (17) for Treatment - same in all PCAs
  scale_shape_manual(values = c("Control" = 16, "Treatment" = 17),
    name = "Condition") +
  scale_color_manual(values = pca_palette, name = "Condition") +
  geom_text_repel(aes(label = name), show.legend = FALSE,
    size = 4, fontface = "bold",
    box.padding = 0.5, bg.color = "white", bg.r = 0.1) +
  labs(title = "PCA of Pectobacterium, Treatment vs Control",
    subtitle = "Filtered and VST normalized data",
    x = paste0("PC1: ", percentVar[1], "% variance"),
    y = paste0("PC2: ", percentVar[2], "% variance")) +
  theme_minimal(base_size = 13) +
  theme(panel.grid.minor = element_blank(),
    plot.title = element_text(face = "bold", size = 14)) +
  coord_fixed()

pca

```



```
# Save the plot in high quality
ggsave(filename = "../results/pca_raw.pdf", plot = pca, width = 11, height = 8.5)
ggsave(filename = file.path(tiff_dir, "pca_raw.tiff"), plot = pca, width = 11, height = 8.5, dpi = 300,
```

2.3. Save workspace

We save the objects in an `.RData` file so we can easily use them in future scripts without running everything again.

```
save(metadata, dds, rawcounts, filtcounts, pca, file = "../results/dds.RData")
```

Session information

Recording the R version and packages used helps make sure this analysis can be reproduced.

```
sessionInfo()
```

```

## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26120)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Spanish_Mexico.utf8  LC_CTYPE=Spanish_Mexico.utf8
## [3] LC_MONETARY=Spanish_Mexico.utf8 LC_NUMERIC=C
## [5] LC_TIME=Spanish_Mexico.utf8
##
## time zone: America/Mexico_City
## tzcode source: internal
##
## attached base packages:
## [1] stats4      stats      graphics  grDevices  utils      datasets  methods
## [8] base
##
## other attached packages:
## [1] ggrepel_0.9.8           lubridate_1.9.5
## [3] forcats_1.0.1           stringr_1.6.0
## [5] dplyr_1.2.1             purrr_1.2.2
## [7] readr_2.2.0             tidyr_1.3.2
## [9] tibble_3.3.1            ggplot2_4.0.3
## [11] tidyverse_2.0.0         DESeq2_1.50.2
## [13] SummarizedExperiment_1.40.0 Biobase_2.70.0
## [15] MatrixGenerics_1.22.0    matrixStats_1.5.0
## [17] GenomicRanges_1.62.1     Seqinfo_1.0.0
## [19] IRanges_2.44.0           S4Vectors_0.48.1
## [21] BiocGenerics_0.56.0      generics_0.1.4
## [23] tximport_1.38.2          viridis_0.6.5
## [25] viridisLite_0.4.3        cowplot_1.2.0
##
## loaded via a namespace (and not attached):
## [1] gtable_0.3.6            xfun_0.56              lattice_0.22-7
## [4] tzdb_0.5.0              vctrs_0.7.3            tools_4.5.2
## [7] parallel_4.5.2          pkgconfig_2.0.3        Matrix_1.7-4
## [10] RColorBrewer_1.1-3      S7_0.2.2               lifecycle_1.0.5
## [13] compiler_4.5.2          farver_2.1.2           textshaping_1.0.5
## [16] codetools_0.2-20        htmltools_0.5.9        yaml_2.3.12
## [19] crayon_1.5.3            pillar_1.11.1          BiocParallel_1.44.0
## [22] DelayedArray_0.36.1     abind_1.4-8            tidyselect_1.2.1
## [25] locfit_1.5-9.12         digest_0.6.39          stringi_1.8.7
## [28] labeling_0.4.3          fastmap_1.2.0          grid_4.5.2
## [31] cli_3.6.5              SparseArray_1.10.10    magrittr_2.0.5
## [34] S4Arrays_1.10.1         withr_3.0.2            scales_1.4.0
## [37] bit64_4.8.0            timechange_0.4.0       rmarkdown_2.31
## [40] XVector_0.50.0          bit_4.6.0              otel_0.2.0
## [43] gridExtra_2.3           ragg_1.5.2             hms_1.1.4
## [46] evaluate_1.0.5          knitr_1.51             rlang_1.2.0
## [49] Rcpp_1.1.1-1.1         glue_1.8.1             vroom_1.7.1
## [52] rstudioapi_0.18.0      R6_2.6.1              systemfonts_1.3.2

```